

Models of Robot Manipulation for a Four-Rotational-Joint Gripper Robot

Devansh Bajwala
Dept. Of Mechanical Engineering
University of Washington
Seattle, United States
db7529@uw.edu

Nathan Ford
Dept. Of Electrical Engineering
University of Washington
Seattle, United States
naford@uw.edu

Anh Nguyen
Dept. Of Electrical and Computer
Engineering
University of Washington
Seattle, United States
anguye22@uw.edu

Abstract— The field of robotics consists of many groundbreaking robotic advancements, but the robot detailed in this report is not one of them. The robot detailed in this report is a four-rotational-joint gripper robot that uses forward kinematics to enable joint-based control and inverse kinematics to enable cartesian control of the end effector. The robot was created to detail the basic principles of 3D robot manipulation. Rather than being a standard 6 degrees of freedom, the robot is instead 4 degrees of freedom. The 4 degrees of freedom present several challenges to the design and implementation process. The reachable workspace is approximated with a Monte-Carlo simulation that calculates whether a desired end effector position is achievable. The robot is modelled in software and created in physical hardware. The GitHub repository detailing the code and files necessary to recreate the robot can be found at this link: <https://github.com/notachevy/ee543-final-project-winter-2025.git>. The project is based upon a template project provided in the GitHub repository found at this link: <https://github.com/HaonanPeng/EE543-Robot-Manipulation-Project.git> [1].

Keywords— Robotics, cartesian control, reachable workspace, rotational joints, link frame assignment, DH table, DH parameters, forward kinematics, inverse kinematics, IKBT, Monte-Carlo simulation.

I. INTRODUCTION

The study of robotics consists of mathematically modelling the positions of and relationships between links and joints to achieve interactions with the environment through an end effector. Robotic arms have a base that defines their set connection to a location and an end effector that allows it to interact with its environment. For our robot, the end effector is set as a gripper that is able to open and close, but many types of end effectors exist.

The current field of robotics is generating significant improvements in the medical field such as surgical robots. Across all fields, machine learning and artificial intelligence (AI) are helping robots interact with their environment by both aiding in physical training and allowing for virtual training through generated scenarios. Some robots focus on modelling human behaviour, but a majority of robots do not attempt to mimic human behavior but rather accomplish a task in an efficient manner. For 3D cartesian movement, a robotic arm is excellent as it usually provides 6 degrees of freedom, a perfect

match to a 3-dimensional space. Advanced robotic arms have 7 degrees of freedom to allow for multiple arm positions, yielding the same end effector position. The advantage of multiple arm positions is that the robot arm can manipulate around obstacles and take predefined paths. The complexity when adding a 7th degree of freedom increases dramatically as the arm will always have multiple solutions, and control code will need to be written to determine the best solution instantaneously and over a pathway.

The more advanced robots will also utilize velocity control using the Jacobian. Velocity control allows for a robot arm to dictate its linear speed with calculations from its angular speed. The advantages to velocity-based control methods are smoother trajectories with fewer overshoots that help protect the robot and its surroundings. The generated trajectories have predefined acceleration paths, and if using a parabolic blend trajectory generation, faster movements from desired locations can be achieved.

II. LINK FRAME ASSIGNMENT AND DH PARAMETERS

To begin work modeling the robot, the graphical and mathematical representations of the robot must be defined. The robot is defined graphically by its link frame assignment and mathematically by its DH parameters, represented in a DH table.

A. Link Frame Assignment

The first step in defining a robot graphically is to define the base frame. The base frame for the robot is defined as the point the center point of the rotational disk meets the base. The +Z axis is defined as a perpendicular line that runs through the center of the rotation disk in the direction of the motor. The next step is to define the location of the final frame (frame five). The final frame is defined as a point in the middle of the grabber with the +Z axis defined as the direction facing outwards from the gripper. The +Z axis assignments can be seen in Fig. 1.

The +X direction of the base frame is set to be parallel to the +X direction of frame one. The +X direction of frame five is defined as parallel to the +X direction of frame four. The rest of the +X directions are assigned using standard DH assignment principles. The +Y directions are omitted from representation as

they can be determined by taking the cross product of +Z cross +X. The +X assignments can be seen in Fig. 1.

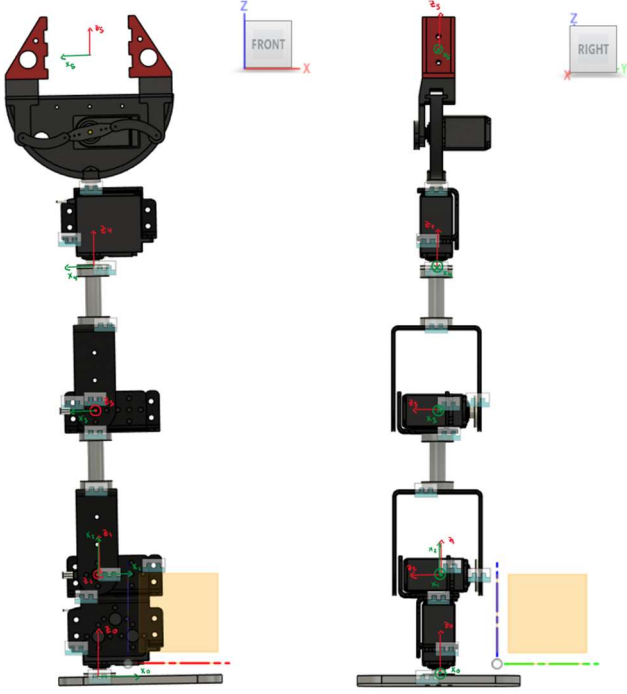


Fig. 1. Link frame assignment overlaid onto 3D model of robot. Perspective cube in top right corner represents world view orientation and direction. Red, green, and blue dashed lines represent world view axis in X, Y, Z format respectively. The yellow plane represents the face of the model (as seen in the perspective cube in the top right corner).

B. DH Table and Parameters

To define a robot mathematically, its graphical representation can be used to define a set of parameters representing the rotational and translational transforms necessary to go from one frame to the next frame. Standard DH parameter assignment is used to get the DH parameters shown in Table I.

TABLE I. DH PARAMETERS

Joint N-1 $\rightarrow$ N	DH Parameters			
	A (mm)	a (deg)	d (mm)	θ (deg)
0 $\rightarrow$ 1	0	0	62.8	θ_1
1 $\rightarrow$ 2	0	90	0	θ_2
2 $\rightarrow$ 3	101	0	0	θ_3
3 $\rightarrow$ 4	0	90	87.5	θ_4
4 $\rightarrow$ 5	0	0	125	0

C. Forward Kinematics Simulation

The previously provided CAD model of the robot arm is excellent at visualizing what the arm will look like but poor at indicating how the arm will move and showing the current state of the robot. To visualize the frames of the arm while it is moving, a forward kinematics simulation models the motion of the arm. The simulation shows the robot arm modelled as simple

links in an empty 3D space. The length of the virtual link joints is proportional to the size of the actual arm links. Each link frame is represented with a red arrow indicating the +Z direction and a blue arrow indicating the +X direction. A snapshot of the simulation can be seen in Fig. 2.

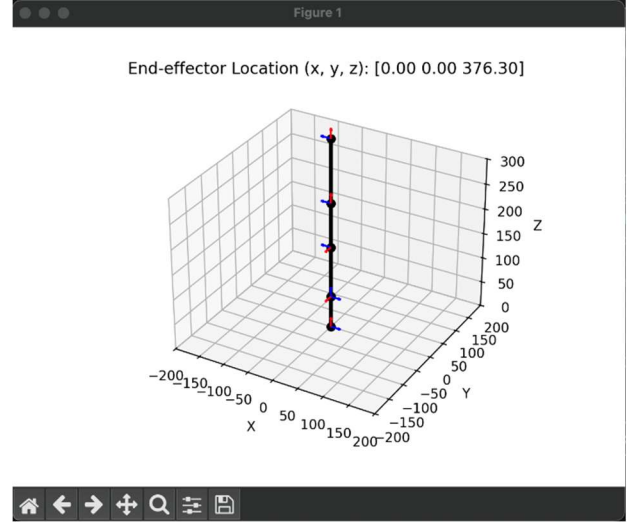


Fig. 2. Forward kinematics simulation showing a virtual representation of the actual arm. The virtual link lengths are proportional to the actual link lengths. Each link frame assignment is shown by indicating the +Z direction with a red arrow and the +X direction with a blue arrow. The robot is shown in the “home” position, or its state when all the servos are homed and their theta values are set to 0.

The simulation is computed using a numeric solving of the forward kinematics equations. The forward kinematics are updated every time a joint angle is changed to display the position of the actual arm in real time. The details of determining the forward kinematics are given in the next section, Forward Kinematics.

III. FORWARD KINEMATICS

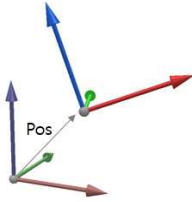
Forward kinematics is the process of applying the state of a robot represented by its rotational joint angles and translational joint distances to determine the orientation and location of the end effector in space. For the four-rotational-joint gripper, only rotational joints are used so only rotational joints will be discussed. Translational joints behave mathematically similar to rotational joints except the “d” value from the DH table is variable while the “ θ ” value is fixed.

Forward kinematics will provide the end effector position given the input joint angles result in a valid orientation. The collection of all valid robot joint angles is called the reachable workspace. The workspace is a complicated 3D space, so a Monte-Carlo simulation is run to visualize and approximation of the workspace. The Monte-Carlo simulation samples many random valid combinations of joint angles to produce a point-cloud of valid end effector positions, giving a visualization of valid end effector positions.

A. Symbolic Forward Kinematics

To computationally save time to calculate the forward kinematics for every instance of the robot, the symbolic representation of the end effector can be found in terms of the

angles of its joints. The symbolic representation begins as a collection of transformation matrices that contain a rotation matrix and a position vector. These transformation matrices use the DH table to generate a transformation matrix for the transformation between every frame, $0 \rightarrow 1$, $1 \rightarrow 2$, $2 \rightarrow 3$, $3 \rightarrow 4$, and $4 \rightarrow 5$. Because each transformation is calculated in the current frame, the total transformation matrix from the base to the end effector, $0 \rightarrow 5$, can be found by post-multiplying the transformation matrices in order. The individual transformation matrices are given in Fig. 3.

$$T = \begin{bmatrix} R & Pos \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} X_x & Y_x & Z_x \\ X_y & Y_y & Z_y \\ X_z & Y_z & Z_z \end{bmatrix} & \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$


$$T_{01} = \begin{bmatrix} \cos(t_1) & -\sin(t_1) & 0 & 0 \\ \sin(t_1) & \cos(t_1) & 0 & 0 \\ 0 & 0 & 1 & 62.8 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{12} = \begin{bmatrix} \cos(t_2) & -\sin(t_2) & 0 & 0 \\ \sin(t_2) \cdot \cos(90) & \cos(90) \cdot \cos(t_2) & -\sin(90) & 0 \\ \sin(90) \cdot \sin(t_2) & \sin(90) \cdot \cos(t_2) & \cos(90) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{23} = \begin{bmatrix} \cos(t_3) & -\sin(t_3) & 0 & 101 \\ \sin(t_3) & \cos(t_3) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{34} = \begin{bmatrix} \cos(t_4) & -\sin(t_4) & 0 & 0 \\ \sin(t_4) \cdot \cos(90) & \cos(90) \cdot \cos(t_4) & -\sin(90) & -87.5 \cdot \sin(90) \\ \sin(90) \cdot \sin(t_4) & \sin(90) \cdot \cos(t_4) & \cos(90) & 87.5 \cdot \cos(90) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{45} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 125 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Fig. 3. Forward kinematics transformation matrices. The transformation matrices represent the rotation matrix (the upper left 3x3 matrix) and the position vector (the upper right 1x3 matrix). From top to bottom, the figures are: a. general transformation matrix form showing the location of the rotation matrix and position vector with a graphical representation of the transformation [2], b. the transformation matrix from frame 0 to frame 1, c. the transformation matrix from frame 1 to frame 2, d. the transformation matrix from frame 2 to frame 3, e. the transformation matrix from frame 3 to frame 4, f. the transformation matrix from frame 4 to 5.

The overall transformation matrix to go from the base frame to the end effector frame, $0 \rightarrow 5$, is quite complicated. The full transformation matrix in Fig. 4 is included for completeness, but for ease of use, the “forward_k.py” file should be run from the project GitHub to acquire the matrix if it is to be used. The full transformation matrix is useful as it allows for a simpler calculation of end effector position given joint angles. Only twelve values need to be calculated given four variables to determine the orientation and location of the end effector. The symbolic representation speeds up calculations as numeric versions of each individual transformation matrix do not need to be multiplied together.

$$\begin{bmatrix} r_{11} & r_{12} & r_{13} & Px \\ r_{21} & r_{22} & r_{23} & Py \\ r_{31} & r_{32} & r_{33} & Pz \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Column 1 $\begin{bmatrix} c_4(-c_1c_2c_3 + c_1s_2s_3) + s_1s_4 \\ -c_1s_4 + c_4(-c_2c_3s_1 + s_1s_2s_3) \\ c_4(-c_2s_3 - c_3s_2) \\ 0 \end{bmatrix}$

Column 2 $\begin{bmatrix} c_4s_1 - s_4(-c_1c_2c_3 + c_1s_2s_3) \\ -c_1c_4 - s_4(-c_2c_3s_1 + s_1s_2s_3) \\ -s_4(-c_2s_3 - c_3s_2) \\ 0 \end{bmatrix}$

Column 3 $\begin{bmatrix} -c_1c_2s_3 - c_1c_3s_2 \\ -c_2s_1s_3 - c_3s_1s_2 \\ c_2c_3 - s_2s_3 \\ 0 \end{bmatrix}$

Column 4 $\begin{bmatrix} -a_3c_1s_2 + d_1(-c_1c_2s_3 - c_1c_3s_2) - l_2(c_1c_2s_3 + c_1c_3s_2) \\ -a_3s_1s_2 + d_1(-c_2s_1s_3 - c_3s_1s_2) - l_2(c_2s_1s_3 + c_3s_1s_2) \\ a_3c_2 + d_1(c_2c_3 - s_2s_3) + l_1 - l_2(-c_2c_3 + s_2s_3) \\ 1 \end{bmatrix}$

Fig. 4. The overall transformation matrix from the base frame to the end effector in general form (top). The individual columns are broken up to enhance

the readability of the symbolic forward kinematics matrix. The columns are numbered 1-4 and read from left to right.

B. Monte-Carlo Simulation

The Monte-Carlo simulation has two uses. The first use is to visualize the workspace and the second is to check if a given point is within or outside the workspace.

Visualizing the workspace is done by taking a large collection of random combinations of joint angles and using the previously discussed symbolic forward kinematics to determine the resulting end effector location. For this simulation, the orientation of the end effector is ignored as the workspace is defined as the region the end effector can reach. Each iteration of the simulation plots one point in 3D space for a valid joint combination. With numerous iterations, a point cloud forms that shows the shape of the 3D workspace. The Monte-Carlo workspace approximation is shown in Fig. 5. The simulation yields a workspace that appears spherical. In practice, the arm is situated on a flat physical surface that prevents the arm from reaching points underneath it. In theory, the arm can reach all points within an outer boundary sphere and outside and inner boundary sphere (components cannot physically overlap).

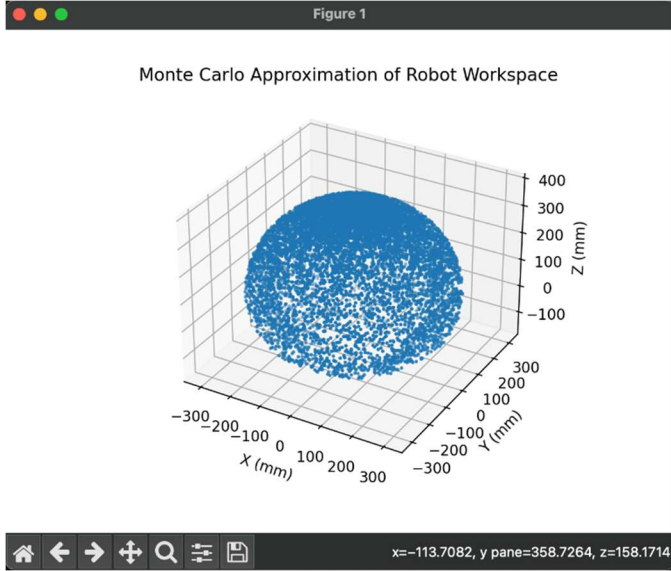


Fig. 5. Monte-Carlo simulation showing a point cloud that approximates the workspace of the robot by plotting the end effector location by plugging in angles into the symbolic forward kinematics and solving using numerous iterations of combinations of valid joint angles.

The second use of the Monte-Carlo simulation is to determine whether a given point is reachable in the workspace. Determining whether a point is within the workspace is helpful in determining the validity of a combination of joint angles to ensure the safety of the robot and the users nearby. To calculate whether the point is within the workspace, the algorithm will sample the nearest points in the cloud and create vectors between the test point and each closest point. The algorithm will then compare the angle between the closest vectors to see whether or not the point is within the tangent of the workspace. To accomplish this, any angle above 85 degrees is assumed close enough to 90 degrees to warrant being within the workspace.

To test the algorithm, three points are given and the algorithm needs to determine whether or not the points are within the workspace. The results are shown in Fig. 6 and Table II.

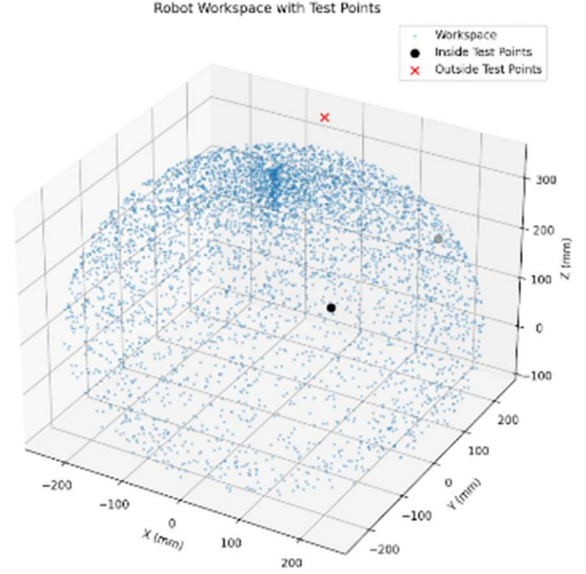


Fig. 6. Monte-Carlo workspace simulation showing test points and whether or not they are within the workspace.

TABLE II. MONTE-CARLO WORKSPACE TEST POINTS

Point	Point Parameters			
	X	Y	Z	Inside or Outside Workspace
1	70	50	100	Inside
2	175	150	400	Outside
3	180	180	200	Inside

IV. INVERSE KINEMATICS AND CARTESIAN SPACE CONTROL

Inverse kinematics is both more complicated and more useful than forward kinematics. Inverse kinematics is the inverse of forward kinematics, where a known end effector position is used to determine the joint angles to produce the desired position and rotation. Unfortunately, for our robot, we have four joints and only three known values, X, Y, and Z. This means our situation is underspecified and there may be multiple solutions. This trouble arises from our robot only having 4 degrees of freedom rather than the standard 6 degrees of freedom associated with X, Y, Z, roll, pitch, and yaw. To keep functionality of our robot, the final rotational joint is perpendicular to the direction of motion of the grabber. This means with one rotational joint; we can control our orientation and do not need to solve for all 6 degrees of freedom.

For our robot, we explored both analytical inverse kinematics and numerical inverse kinematics to achieve

cartesian control rather than joint control. The analytical inverse kinematics is too complicated to be handled by solvers such as Wolfram-Alpha, so it needed to be attempted manually. Our attempt to solve the inverse kinematics by hand was unsuccessful and we ended up using a prebuilt IKBT library made by Professor Blake Hannaford to compute the analytical inverse kinematics solution [3]. For the actual implementation, numeric inverse kinematics was solved using an optimizer to minimize the error between desired and actual end effector position with respect to joint angles.

With solved inverse kinematics, the end goal of controlling the end effector in cartesian space can be achieved. Cartesian control is more intuitive for operators than joint control, so it is preferred even though it is more computationally difficult.

A. Analytical Inverse Kinematics

To quickly solve the joint angles required, an analytical solution will yield symbolic representations of the joint angles. These symbolic representations are significantly faster to compute than a numeric approximation of joint angles. To compute the analytical solution to the inverse kinematics, the forward kinematics solution from the base to the end effector is taken and solved for the joint angles.

To solve for the joint angles, there need to be known values in the transformation matrix. Unfortunately, we are assuming for our cartesian control that we do not have control over the orientation of the end effector directly. To orient our end effector, we are using the rotational joint connected to the gripper, joint 4. Having control over X, Y, Z, and roll (in the direction of +Z axis) means our robot arm only has 4 degrees of freedom rather than the standard 6 degrees of freedom in 3D space. Because we are solving our inverse kinematics for the cartesian control, we are assuming the only three known values are X, Y, and Z. When solving for the joint angles, joint 4 does not impact the end effector position, so we only need three equations to solve for joints 1, 2, and 3. The three equations that dependent upon joints 1, 2, and 3 conveniently are the equations for X, Y, and Z. With these equations to solve determined, we began the task of isolating for each joint angle. Unfortunately, the joint angle dependencies made it too difficult to isolate the variables by hand. With the equations isolated, we tried an online solver to solve for the joint angles, but the system of equations proved too difficult for Wolfram Alpha to solve as well. The manual and Wolfram Alpha attempts to solve for the joint angles are shown in Fig. 7.

Equation 1: $212.5[-c_1c_2s_3 - s_2c_1c_2] + 101c_1c_2 = r_{03}$
Equation 2: $212.5s_1s_2c_3 + 212.5s_1c_2 + 101s_1c_2 = r_{13}$
Equation 3: $-212.5(s_2s_3 + c_2c_3) + 101s_2 + 62.8 = r_{23}$
Equation 4: $1 = r_{33}$

Equation 2: $s_1[212.5s_2c_3 + 212.5c_2 + 101c_2] = r_{13}$
 $s_1 = r_{13} / (212.5s_2c_3 + 212.5c_2 + 101c_2)$

Equation 1: $212.5[-c_1c_2s_3 - s_2c_1c_2] + 101c_1c_2 = r_{03}$
 $-212.5c_1 \sin(\theta_2 + \theta_3) + 101c_1c_2 = r_{03}$
 $c_1 = r_{03} / (-212.5 \sin(\theta_2 + \theta_3) + 101c_2)$

Equation 3: $-212.5(s_2s_3 + c_2c_3) + 101s_2 + 62.8 = r_{23}$
 $-212.5 \cos(\theta_2 - \theta_3) + 101s_2 + 62.8 = r_{23}$

Equation 2: $s_1 = r_{13} / (212.5s_2c_3 + 313.5c_2)$
Equation 1: $s_1 = r_{03} / (-212.5 \sin(\theta_2 + \theta_3) + 101c_2)$

Equation 3: $-212.5 \cos(\theta_2 - \theta_3) + 101s_2 + 62.8 = r_{23}$
 $\cos \theta_2 = (r_{23} + 212.5 \cos \theta_3 - 101s_2 - 62.8) / 212.5$
 $\Rightarrow \theta_2 - \theta_3 = \arccos((r_{23} + 212.5 \cos \theta_3 - 101s_2 - 62.8) / 212.5)$

Equation 1: $212.5[-c_1c_2s_3 - s_2c_1c_2] + 101c_1c_2 = r_{03}$
 $-212.5c_1(c_2s_3 + s_2c_3) + 101c_1c_2 = r_{03}$
 $-212.5c_1 \sin(\theta_2 + \theta_3) + 101c_1c_2 = r_{03}$
 $\theta_2 + \theta_3 = \arcsin((r_{03} - 101c_2) / (-212.5))$

solve	$101 \cos(x) \cos(y) + 212.5(-\cos(z) \sin(y) - \cos(x) \cos(y) \sin(z)) = a$
	$101 \cos(y) \sin(x) + 212.5(\cos(y) \sin(x) + \sin(y) \sin(z)) = b$
	$62.8 + 101 \sin(y) - 212.5(\cos(y) \cos(z) + \sin(y) \sin(z)) = c$
	$1 = d$

Fig. 7. Manual (top) and Wolfram Alpha (bottom) attempts at solving for the joint angles 1, 2, and 3. The manual attempt details attempts at solving for the joint angles theta 1 with theta 2 and theta 3 dependencies. Theta 2 and theta 3 could not be isolated from each other. The Wolfram Alpha attempt at solving did not yield a solution or formal attempt as the complexity is greater than it can compute.

Because the manual analytical inverse kinematics was unsuccessful, we turned to an existing library to assist with solving our inverse kinematics. The library is based upon Professor Blake Hannaford's work in solving inverse kinematics with symbolic behaviour trees (called IBKT in the given GitHub link above). The process involves finding closed-form solutions to inverse kinematics problems with actions in behavior trees. The library generates a report called "IK_Report_Template" that is visible on our project GitHub repository. Fig. 8 Shows the joint angle solutions while Fig. 9 shows the dependencies between joint angles.

$$\theta_{1v1} = \text{atan}_2(r_{23}, r_{13})$$

$$\theta_{1v2} = \text{atan}_2(r_{23}, r_{13})$$

$$\theta_{1v3} = \text{atan}_2(r_{23}, r_{13})$$

$$\theta_{1v4} = \text{atan}_2(r_{23}, r_{13})$$

$$\begin{aligned}\theta_{2v1} &= \arcsin\left(\frac{Pz - d_1r_{33} - l_1 - l_2r_{33}}{a_3}\right) \\ \theta_{2v2} &= \arcsin\left(\frac{Pz - d_1r_{33} - l_1 - l_2r_{33}}{a_3}\right) \\ \theta_{2v3} &= \arcsin\left(\frac{Pz - d_1r_{33} - l_1 - l_2r_{33}}{a_3}\right) \\ \theta_{2v4} &= \arcsin\left(\frac{Pz - d_1r_{33} - l_1 - l_2r_{33}}{a_3}\right)\end{aligned}$$

$$\begin{aligned}\theta_{3e1} &= \operatorname{atan2}(r_{13} \cos(\theta_{1s1}) \cos(\theta_{2v1}) + r_{23} \sin(\theta_{1s1}) \cos(\theta_{2v1}) + r_{33} \sin(\theta_{2v1}), r_{13} \sin(\theta_{1s1}) \cos(\theta_{2v1}) + r_{23} \sin(\theta_{1s1}) \sin(\theta_{2v1}) \\ \theta_{3e2} &= \operatorname{atan2}(r_{13} \cos(\theta_{1s2}) \cos(\theta_{2v2}) + r_{23} \sin(\theta_{1s2}) \cos(\theta_{2v2}) + r_{33} \sin(\theta_{2v2}), r_{13} \sin(\theta_{1s2}) \cos(\theta_{2v2}) + r_{23} \sin(\theta_{1s2}) \sin(\theta_{2v2}) \\ \theta_{3e3} &= \operatorname{atan2}(r_{13} \cos(\theta_{1s2}) \cos(\theta_{2v3}) + r_{23} \sin(\theta_{1s2}) \cos(\theta_{2v3}) + r_{33} \sin(\theta_{2v3}), r_{13} \sin(\theta_{1s2}) \cos(\theta_{2v3}) + r_{23} \sin(\theta_{1s2}) \sin(\theta_{2v3}) \\ \theta_{3e4} &= \operatorname{atan2}(r_{13} \cos(\theta_{1s1}) \cos(\theta_{2v4}) + r_{23} \sin(\theta_{1s1}) \cos(\theta_{2v4}) + r_{33} \sin(\theta_{2v4}), r_{13} \sin(\theta_{1s1}) \cos(\theta_{2v4}) + r_{23} \sin(\theta_{1s1}) \sin(\theta_{2v4}) \\ \theta_{4v1} &= \operatorname{atan2}(r_{11} \sin(\theta_{1s1}) - r_{21} \cos(\theta_{1s1}), r_{12} \sin(\theta_{1s1}) - r_{22} \cos(\theta_{1s1})) \\ \theta_{4v2} &= \operatorname{atan2}(r_{11} \sin(\theta_{1s2}) - r_{21} \cos(\theta_{1s2}), r_{12} \sin(\theta_{1s2}) - r_{22} \cos(\theta_{1s2})) \\ \theta_{4v3} &= \operatorname{atan2}(r_{11} \sin(\theta_{1s2}) - r_{21} \cos(\theta_{1s2}), r_{12} \sin(\theta_{1s2}) - r_{22} \cos(\theta_{1s2})) \\ \theta_{4v4} &= \operatorname{atan2}(r_{11} \sin(\theta_{1s1}) - r_{21} \cos(\theta_{1s1}), r_{12} \sin(\theta_{1s1}) - r_{22} \cos(\theta_{1s1}))\end{aligned}$$

$$\begin{aligned}\theta_{23v1} &= \theta_{2v1} + \theta_{3v1} \\ \theta_{23v2} &= \theta_{2v2} + \theta_{3v2} \\ \theta_{23v3} &= \theta_{2v3} + \theta_{3v3} \\ \theta_{23v4} &= \theta_{2v4} + \theta_{3v4}\end{aligned}$$

Fig. 8. IKBT inverse kinematics solutions for joints 1 through 4. Solutions found using IKBT library [3]. More legible versions of the solutions can be found on the project GitHub under the file: “ik_report_template.” From theta 1 to theta 4 and then theta 23, the methods used to solve for each joint angle are as follows: arctan2(y, x), arcsin(y), arctan2(y, x), arctan2(y, x), and algebra.

In the solutions, some joint angles have multiple solutions and are dependent upon the multiple solutions of other joint angles. To clarify the connections between multiple joint angle solutions and their dependencies, a dependency graph is provided in Fig. 9.

Edge:th_23 depends on: th_3 (th_23-->th_3) Edge:th_3 depends on: th_1 (th_3-->th_1)
Edge:th_23 depends on: th_2 (th_23-->th_2) Edge:th_4 depends on: th_1 (th_4-->th_1)
Edge:th_3 depends on: th_2 (th_3-->th_2)

Fig. 9. IKBT graphical edge connections showing theta dependencies where “th” represents the joint angle and “-->” represents the direction of the dependency.

Unfortunately, because the IKBT solver uses the rotation matrix ($r_{11}:r_{33}$), the solutions cannot be implemented into the code. The implementation of the robot does not have the 6 degrees of freedom required to utilize the rotation matrix fully. Because of this, a numeric approach to solving the inverse kinematics was used in the code.

B. Numerical Inverse Kinematics

For simplicity’s sake, a numerical inverse kinematics solver is significantly easier from an implementation standpoint. From a computation viewpoint however, numerical inverse kinematics is significantly slower than analytical inverse kinematics.

To accomplish the numerical inverse kinematics, the scipy optimize package was used to minimize the error between the current and desired end effector location. The algorithm tries different combinations of joint angles until it converges to the

solution with the lowest error. The final joint angles are determined to be the estimated solution to the inverse kinematics equation. To ensure convergence, the SLSQP optimization method was used. This method will allow convergence on desired locations outside of the workspace and prevent the arm controls from crashing. The optimizer uses normalized joint angles, so the bounds of the optimization have to be set to ensure the optimizer only tries valid joint angles. The optimization works best in small steps to avoid alternate solutions, so the inverse kinematics function is run several times on intermediate points rather than just once from a desired starting and stopping point. The smaller steps allow for smoother and more consistent motion from the arm. The code, including the optimization, is shown in Fig. 10.

```
def InverseK(self, desired_pos, initial_guess=None, max_iterations=100):
    """
    Solve inverse kinematics for desired EE position.
    """
    # Setting Initial Guess as the current Joint Pose of the Arm
    if initial_guess is None:
        initial_guess = self.robotstate_joint_poses.copy()
    # Desired Position Format declared for me to input
    desired_pos = np.array(desired_pos, dtype=np.float64).flatten()

    def objective(joint_angles):
        # Step 1: Compute Forward Kinematics
        FK_N = self.ForwardK_N(joint_angles)
        # Step 2: Extract the current end-effector position from 4th column of 1st 3 rows
        current_pos = FK_N[:3, 3]
        # Step 3: Calculate the Euclidean Distance between current position and desired position
        error = np.linalg.norm(current_pos - desired_pos)
        return error

    # Clipping the servo angles from their min-max ranges for the IK to give correct angles within the limits
    bounds = [(self.servo_angle_min, self.servo_angle_max) for _ in range(self.joint_num)]

    # Minimize - Generic optimization routine as in manual
    res = minimize(objective, initial_guess, method='SLSQP', bounds=bounds, options={'maxiter': max_iterations, 'ftol': 1e-6})

    if res.success:
        limit_angles = np.clip(res.x, self.servo_angle_min, self.servo_angle_max)
        return limit_angles
    else:
        print(f"IK failed: {res.message}")
        return None
```

Fig. 10. Code for the numeric inverse kinematics solver. The minimize function is used to iteratively determine numeric approximations of joint angles required to reach the desired end effector position. The numeric approximation will converge for any point entered, within or outside the workspace.

C. Cartesian Control

The goal of the project was to use a computer keyboard to control the location of the end effector in cartesian space. The end effector could have its orientation changed with the rotation of joint 4 and manipulate objects by opening and closing the gripper. With this functionality, we attempted picking up a small cylinder and moving it to another location.

During our demonstration, we were able to use a predefined trajectory to have our robot pick up a small object and drop it in a designated bin in a different location. Both the start and end locations are given in [X, Y, Z] format. The robot moves from the home position, [0, 0, 376.30] to the desired pick-up location and closes the gripper. Once the gripper is closed, the robot moves from the pick-up location to the desired drop-off location. The robot then opens the gripper at the target destination. A video demonstration of this can be found in the GitHub repository for the project.

The control code was difficult to implement but simple conceptually. The control code workflow is depicted in Fig. 11 and shows the process for cartesian control with user interaction.

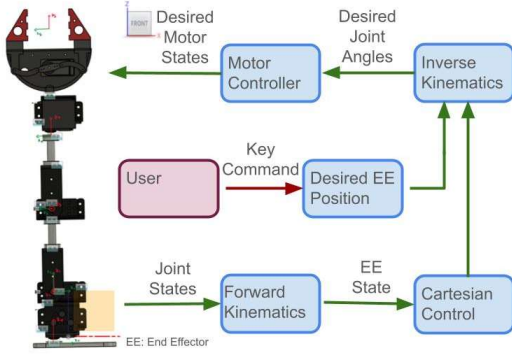


Fig. 11. System workflow where blocks show processes and arrows show information transfer.

Due to human error and potential miscalculations, we were unable to solve the symbolic inverse kinematics analytically by hand. Therefore, we used an external IKBT library instead to figure out the results. The library yielded results that required the rotation matrix, something our 4 degrees of freedom could not manage. Because of this, we were unable to implement the symbolic inverse kinematics into our control code. With only a functional numeric inverse kinematics solver, the runtimes of the numeric and analytical solvers cannot be compared experimentally. Ideally, the analytical solve would be faster as a closed-form solution is known and requires minimal-compute time. With just the numeric solver, undesirable behavior can be seen in how point queries can take up to twice as long as other queries. Query time inconsistency is undesirable in ensuring smooth and accurate robot manipulation. The numerical inverse kinematics runtimes for the three previously discussed points is provided in Table III.

TABLE III. ANALYTICAL VS. NUMERIC INVERSE KINEMATICS SOLVE TIMES

Point	Inverse Kinematics Method	
	Analytical Solving Time (ms)	Numeric Solving Time (ms)
1	N/A	5.52
2	N/A	4.76
3	N/A	10.8

V. CONCLUSION AND FURTHER WORK

The goal of the project was to use a computer keyboard to control the location of the end effector in cartesian space – this

goal was achieved. Additional goals were attempted but not completed. These areas for further work include determining an analytical solution manually, completing velocity-based control using the Jacobian, and avoiding harmful singularities. All areas are within the capabilities of the team but not feasible within the time constraints given.

After completion of the basic robot, additional uses of the robot have been devised by team members and two main pathways have been determined: Computer Vision and Autonomous Operation and Teleoperation.

The current functionality of the robot arm allows for movement to specified $[X, Y, \text{ and } Z]$ coordinates. An application that takes advantage of this would be to implement computer vision to identify the target object and target location (in our demo, a waste basket). With the object and destination identified, their locations could be determined from the camera's frame. Using similar frame transformations to the ones described in this paper, a transformation(s) could be calculated to represent the object and destination in the base frame of the robot to allow for end effector interaction.

Along with the completion of this robot, five additional robot arms were prepared simultaneously by additional teams. With their creation of additional arms, one team incidentally created the same physical robot arm as the one described in this paper. With two identical robots, teleoperation becomes straightforward as the joint angle mapping becomes 1:1. One robot arm is able to directly mirror the other. With mirroring robot arms, a remotely operated arm can be controlled by a human user with deeper critical thinking than can be programmed. The human operator allows for completion of complex and novel tasks.

Overall, the development of a basic four-rotational-joint gripper robot allowed for significant development of robotics skills and the showcase of robot-environment interactions. The team is interested in the proposed future works and will continue their work in implementing advances in their robotic endeavors.

REFERENCES

- [1] H. Peng, EE543 Robotic Manipulation Project, Seattle: GitHub, 2025.
- [2] CoppeliaSim Robotics, "CoppeliaSim User Manual on Positions and Orientations," [Online]. Available: <https://manual.coppeliarobotics.com/index.html>. [Accessed 19 March 2025].
- [3] B. Hannaford and D. Zhang, IKBT: Solving Symbolic Inverse Kinematics with Behavior Tree, Seattle: AI Access Foundation, 2019.